

Exp:3 UCS

Program:

```
import heapq

def uniform_cost_search(graph, start, goal):

    priority_queue = [(0, start, [start])] # (cost, node, path)
    visited = set()

    while priority_queue:

        cost, node, path = heapq.heappop(priority_queue)

        if node in visited:

            continue

        visited.add(node)

        if node == goal:

            return cost, path

        for neighbour, edge_cost in graph[node]:

            if neighbour not in visited:

                heapq.heappush(priority_queue,

                               (cost + edge_cost,

                                neighbour,

                                path + [neighbour]))

    return float("inf"), []

graph = {
    'A': [('B', 2), ('C', 4)],
    'B': [('D', 3), ('E', 5)],
    'C': [('F', 2)],
    'D': [('G', 6)],
    'E': [('G', 1)],
    'F': [('G', 4)],
    'G': []
}
```

```
start = 'A'

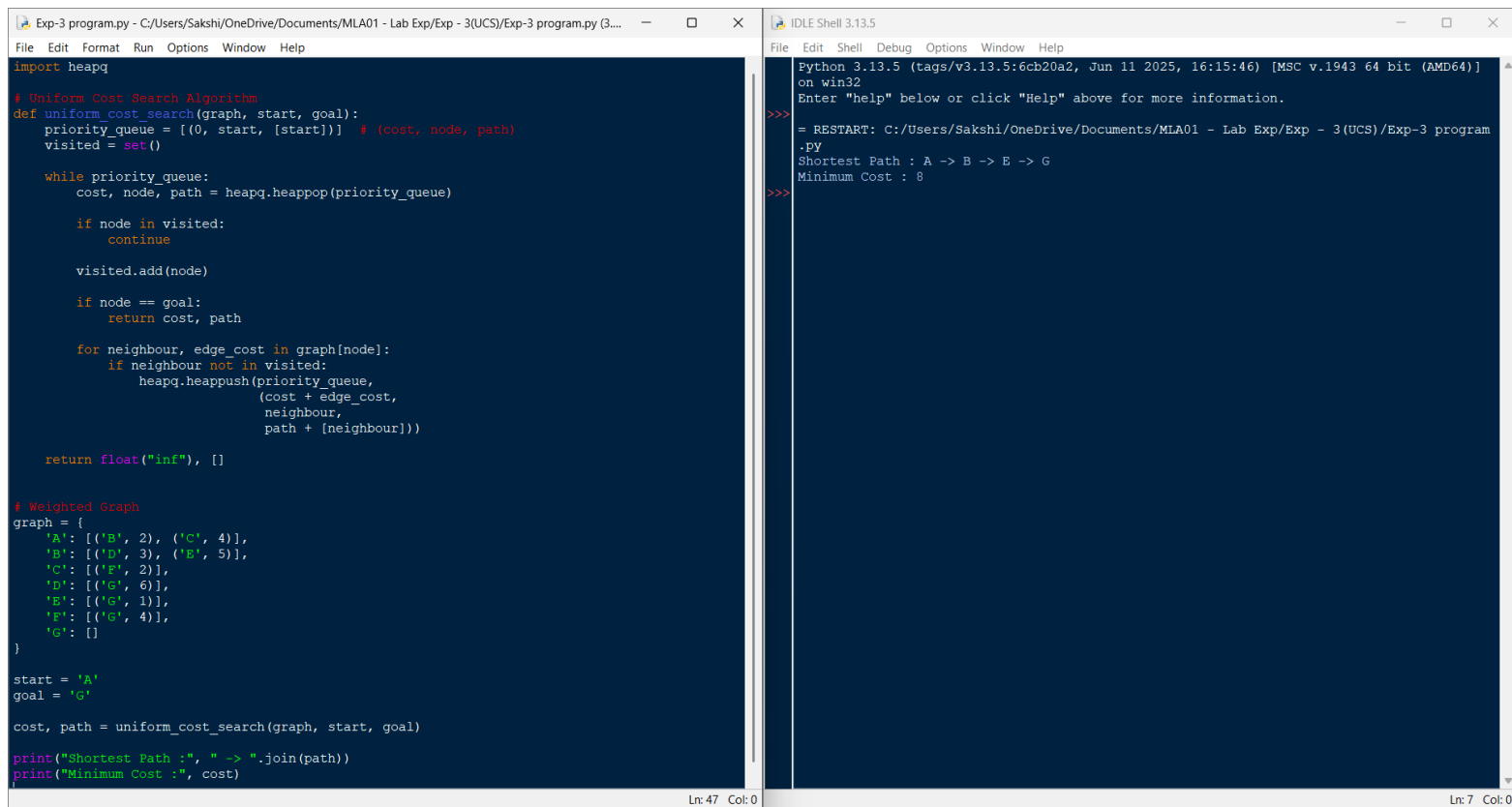
goal = 'G'

cost, path = uniform_cost_search(graph, start, goal)

print("Shortest Path :", " -> ".join(path))

print("Minimum Cost :", cost)
```

Output:



The image shows a screenshot of a code editor with two panes. The left pane displays a Python script for a Uniform Cost Search algorithm. The script defines a function `uniform_cost_search` that takes a graph, start node, and goal node as input. It uses a priority queue and a visited set to find the shortest path. The graph is defined as a weighted graph with nodes A through F. The start node is 'A' and the goal node is 'G'. The script prints the shortest path and the minimum cost.

```
import heapq

# Uniform Cost Search Algorithm
def uniform_cost_search(graph, start, goal):
    priority_queue = [(0, start, [start])] # (cost, node, path)
    visited = set()

    while priority_queue:
        cost, node, path = heapq.heappop(priority_queue)

        if node in visited:
            continue

        visited.add(node)

        if node == goal:
            return cost, path

        for neighbour, edge_cost in graph[node]:
            if neighbour not in visited:
                heapq.heappush(priority_queue,
                               (cost + edge_cost,
                                neighbour,
                                path + [neighbour]))

    return float("inf"), []

# Weighted Graph
graph = {
    'A': [('B', 2), ('C', 4)],
    'B': [('D', 3), ('E', 5)],
    'C': [('F', 2)],
    'D': [('G', 6)],
    'E': [('G', 1)],
    'F': [('G', 4)],
    'G': []
}

start = 'A'
goal = 'G'

cost, path = uniform_cost_search(graph, start, goal)

print("Shortest Path :", " -> ".join(path))
print("Minimum Cost :", cost)
```

The right pane shows the output of the script in an IDLE shell. The output is:

```
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Sakshi/OneDrive/Documents/MLA01 - Lab Exp/Exp - 3 (UCS)/Exp-3 program
.PY
Shortest Path : A -> B -> E -> G
Minimum Cost : 8
>>>
```